**Key Characteristics of Memory Systems:**

| | |
|---|---|
| **Location** | **Performance** |
| Internal (e.g., processor registers, cache, main memory) | Access time |
| External (e.g., optical disks, magnetic disks, tapes) | Cycle time |
| **Capacity** | Transfer rate |
| Number of words | **Physical Type** |
| Number of bytes | Semiconductor |
| **Unit of Transfer** | Magnetic |
| Word | Optical |
| Block | Magneto-optical |
| **Access Method** | **Physical Characteristics** |
| Sequential | Volatile/nonvolatile |
| Direct | Erasable/nonerasable |
| Random | **Organization** |
| Associative | Memory modules |

**Capacity:**

An obvious characteristic of memory is its capacity. For internal memory, this is typically expressed in terms of bytes (1 byte = 8 bits) or words. Common word lengths are 8, 16, and 32 bits.

**Unit of transfer:**

A related concept is the unit of transfer. For internal memory, the unit of transfer is equal to the number of electrical lines into and out of the memory module. This may be equal to the word length, but is often larger, such as 64, 128, or 256 bits. To clarify this point, consider three related concepts for internal memory:

- **Word:** The "natural" unit of organization of memory. The size of a word is typically equal to the number of bits used to represent an integer and to the instruction length. Unfortunately, there are many exceptions. For example, the CRAY C90 (an older model CRAY supercomputer) has a 64-bit word length but uses a 46-bit integer representation. The Intel x86 architecture has a wide variety of instruction lengths, expressed as multiples of bytes, and a word size of 32 bits.
- **Addressable units:** In some systems, the addressable unit is the word. However, many systems allow addressing at the byte level. In any case, the relationship between the length in bits A of an address and the number N of addressable units is $2^A = N$.
- **Unit of transfer:** For main memory, this is the number of bits read out of or written into memory at a time. The unit of transfer need not equal a word or an addressable unit. For external memory, data are often transferred in much larger units than a word, and these are referred to as blocks.

CT**Access Method:** Another distinction among memory types is the method of accessing units of data. These include the following:

- **Sequential access:** Memory is organized into units of data, called records. Access must be made in a specific linear sequence. Stored addressing information is used to separate records and assist in the retrieval process. A shared read–write mechanism is used, and this must be moved from its current location to the desired location, passing and rejecting each intermediate record. Thus, the time to access an arbitrary record is highly variable. Example: Magnetic Tape (used in backups and archives), To read record #80, the read/write head starts from the beginning and passes records 1 to 79 first.
- **Direct access:** As with sequential access, direct access involves a shared read–write mechanism. However, individual blocks or records have a unique address based on physical location. Access is accomplished by direct access to reach a general vicinity plus sequential searching, counting, or waiting to reach the final location. Again, access time is variable. Example: Hard Disk Drive (HDD)**,** The disk arm moves directly to the desired track (direct access), and then the disk spins (sequentially) until the required sector is under the head.

- **Random access:** Each addressable location in memory has a unique, physically wired-in addressing mechanism. The time to access a given location is independent of the sequence of prior accesses and is constant. Thus, any location can be selected at random and directly addressed and accessed. Main memory and some cache systems are random access. Every memory cell has a unique electronic address. Example: RAM (Main Memory) or Cache Memory**,** to read location 2048, the system directly goes to that address — no need to pass through other locations.

**Performance:** From a user's point of view, the two most important characteristics of memory are capacity and performance. Three performance parameters are used:

- **Access time (latency):** For random-access memory, this is the time it takes to perform a read or write operation, that is, the time from the instant that an address is presented to the memory to the instant that data have been stored or made available for use. For non-random-access memory, access time is the time it takes to position the read–write mechanism at the desired location.
- **Memory cycle time:** This concept is primarily applied to random-access memory and consists of the access time plus any additional time required before a second access can commence. This additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively. Note that memory cycle time is concerned with the system bus, not the processor.
- **Transfer rate:** This is the rate at which data can be transferred into or out of a memory unit. For random-access memory, it is equal to 1/(cycle time). For non-random-access memory, the following relationship holds:

$$T_n = T_A + \frac{n}{R}$$

where

$T_n$ = Average time to read or write $n$ bits
$T_A$ = Average access time
$n$ = Number of bits
$R$ = Transfer rate, in bits per second (bps)

**Example 1:**

Suppose we have the following data: Average access time TA=8 ms=0.008 s, Transfer rate R=1 Mbps=1,000,000 bps, Number of bits to read n=800,000 bits. Calculate average time.

$$T_n = T_A + \frac{n}{R}$$

$$T_n = 0.008 + \frac{800{,}000}{1{,}000{,}000}$$

$$T_n = 0.008 + 0.8$$

$$T_n = 0.808 \text{ seconds}$$

**Example 2:**

CT Let's say you're uploading a 2 GB video file from your laptop to cloud storage (like Google Drive or AWS S3) with Transfer rate (R) = 50 Mbps, Average access latency ($T_A$) = 120 ms for Time for handshake, connection setup, DNS lookup, and request acknowledgment, then calculate average time to read and write. **(H.W.)**

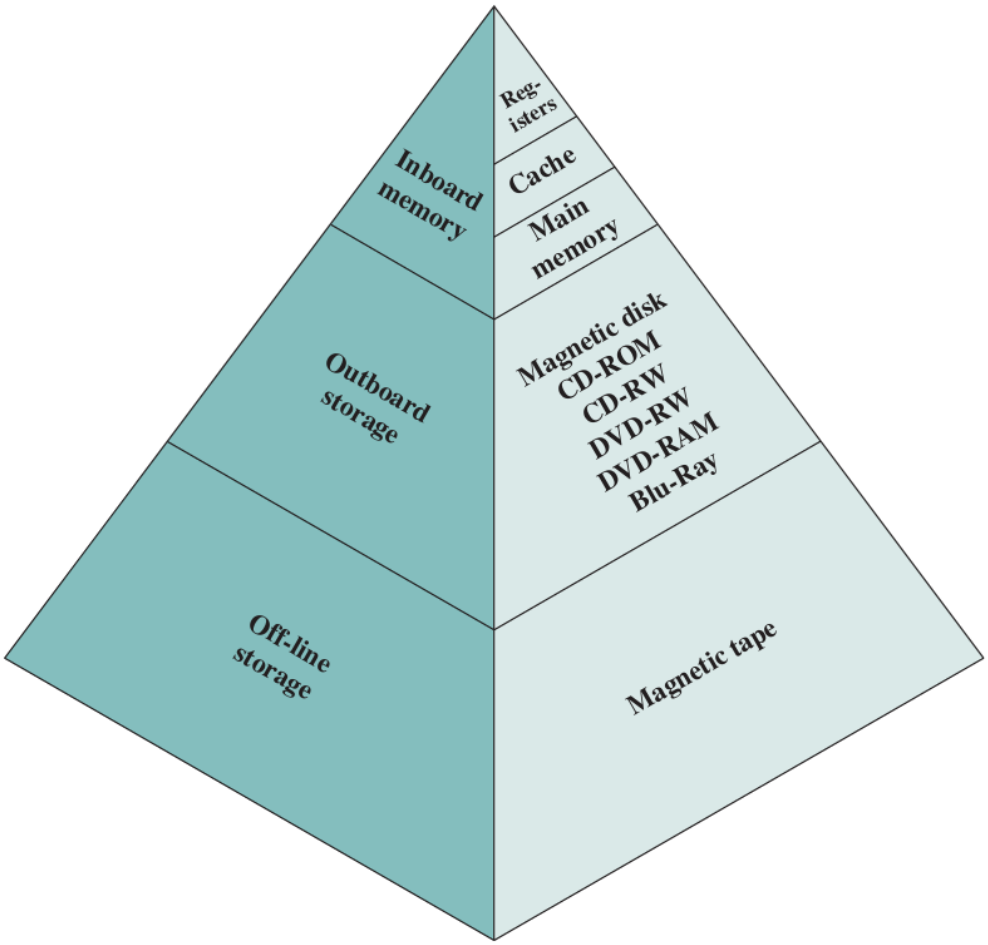
Given,
file size= 2GB =2*1024MB = 2048MB = 2048*8 Mb =16384Mb
R= 50 Mbps
TA=120ms=0.12s
So,
Tn=TA+(File size/R)
=0.12+(16384/50)
=0.12+327.68
=327.8s

Prove:
−H2
−H2
−H2
−H1
(1−H2
−H2
−H2
so,
AMAT=T1+(1-H1)T2+(1−H1

[Proved]

**Memory Hierarchy:**

CT




As one goes down the hierarchy, the following occur:

a. Decreasing cost per bit

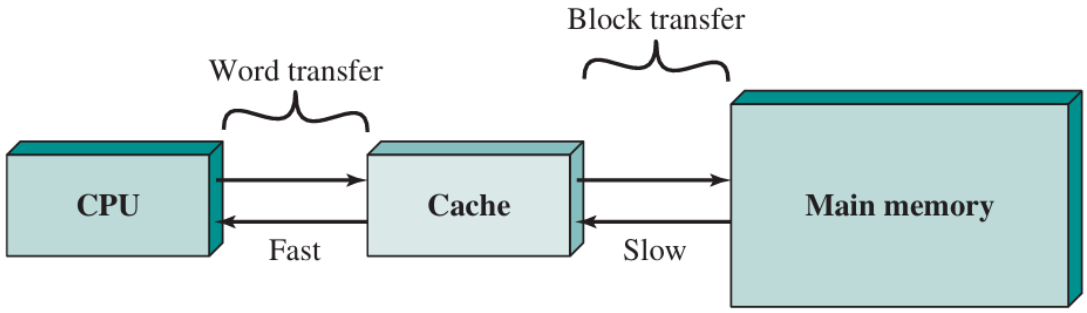
b. Increasing capacity

c. Increasing access time

d. Decreasing frequency of access of the memory by the processor

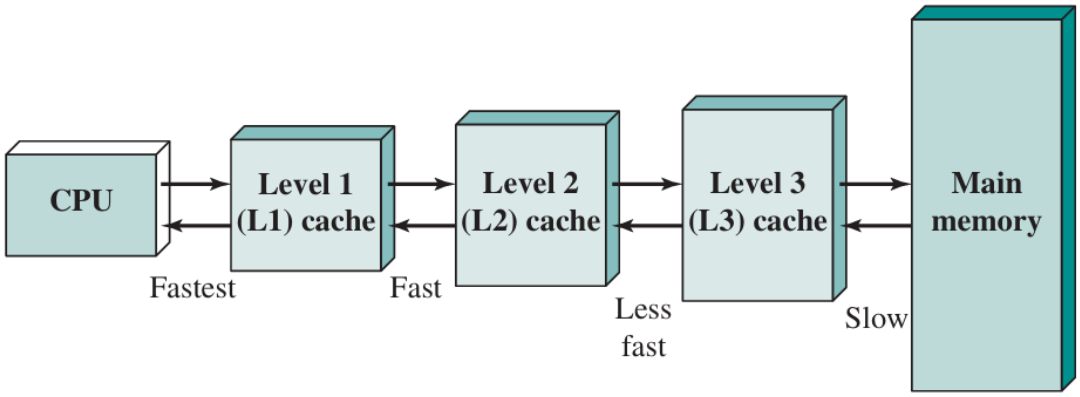
**Cache Memory Principle:**

The cache contains a copy of portions of main memory. When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache. If so, the word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of words, is read into the cache and then the word is delivered to the processor. Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block. Figure 3(b) depicts the use of multiple levels of cache. The L2 cache is slower and typically larger than the L1 cache, and the L3 cache is slower and typically larger than the L2 cache.




(a) Single cache

CT




(b) Three-level cache organization

**Figure 4.3** Cache and Main Memory

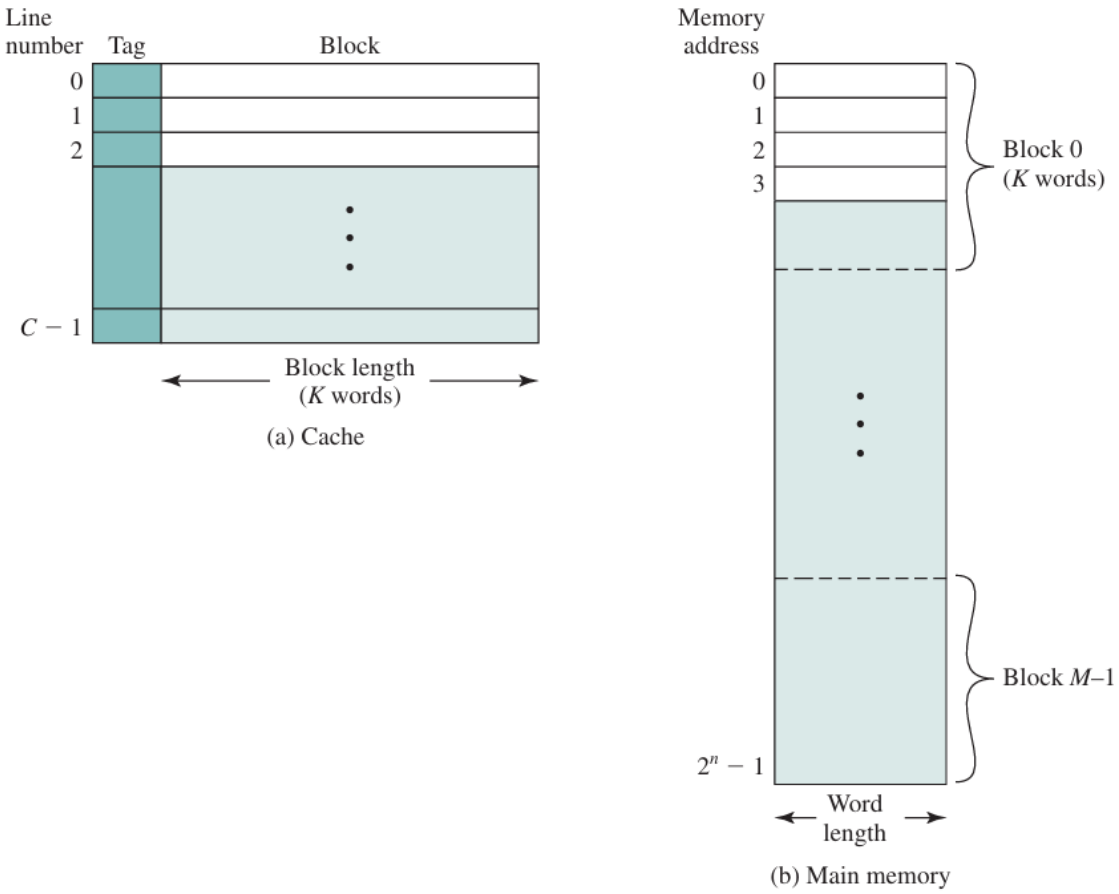



**Figure 4.4** Cache/Main Memory Structure

Figure 4.4 depicts the structure of a cache/main-memory system. Main memory consists of up to $2^n$ addressable words, with each word having a unique $n$-bit address. For mapping purposes, this memory is considered to consist of a number of fixed-length blocks of $K$ words each. That is, there are $M = 2^n/K$ blocks in main memory. The cache consists of $m$ blocks, called **lines**.[3] Each line contains $K$ words, plus a tag of a few bits. Each line also includes control bits (not shown), such as a bit to indicate whether the line has been modified since being loaded into the cache. The length of a line, not including tag and control bits, is the **line size**. The line size may be as small as 32 bits, with each "word" being a single byte; in this case the line size is 4 bytes. The number of lines is considerably less than the number of main memory blocks ($m \ll M$). At any time, some subset of the blocks of memory resides in lines in the cache. If a word in a block of memory is read, that block is transferred to one of the lines of the cache. Because there are more blocks than lines, an individual line cannot be uniquely and permanently dedicated to a particular block. Thus, each line includes a **tag** that identifies which particular block is currently being stored. The tag is usually a portion of the main memory address, as described later in this section.

Figure 4.5 illustrates the read operation. The processor generates the read address (RA) of a word to be read. If the word is contained in the cache, it is delivered to the processor. Otherwise, the block containing that word is loaded into the cache, and the word is delivered to the processor. Figure 4.5 shows these last two operations occurring in parallel and reflects the organization shown in Figure 4.6, which is typical of contemporary cache organizations. In this organization, the cache connects to the processor via data, control, and address lines. The data and address lines also attach to data and address buffers, which attach to a system bus from

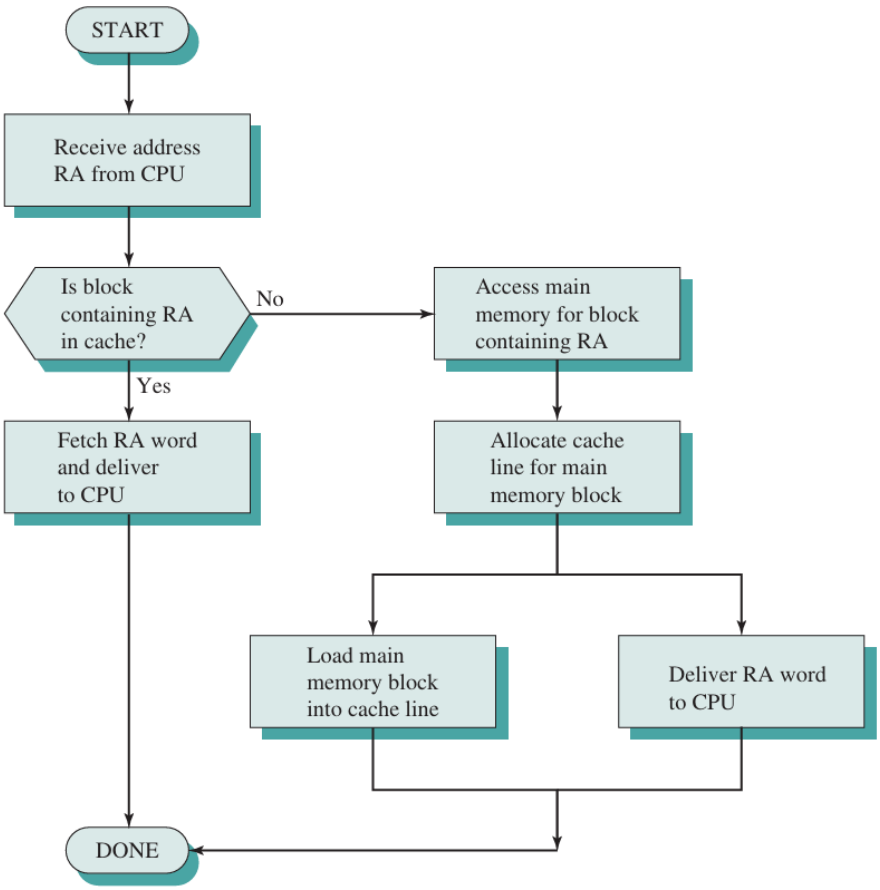



**Figure 4.5** Cache Read Operation

which main memory is reached. When a cache hit occurs, the data and address buffers are disabled and communication is only between processor and cache, with no system bus traffic. When a cache miss occurs, the desired address is loaded onto the system bus and the data are returned through the data buffer to both the cache and the processor. In other organizations, the cache is physically interposed between the processor and the main memory for all data, address, and control lines. In this latter case, for a cache miss, the desired word is first read into the cache and then transferred from cache to processor.

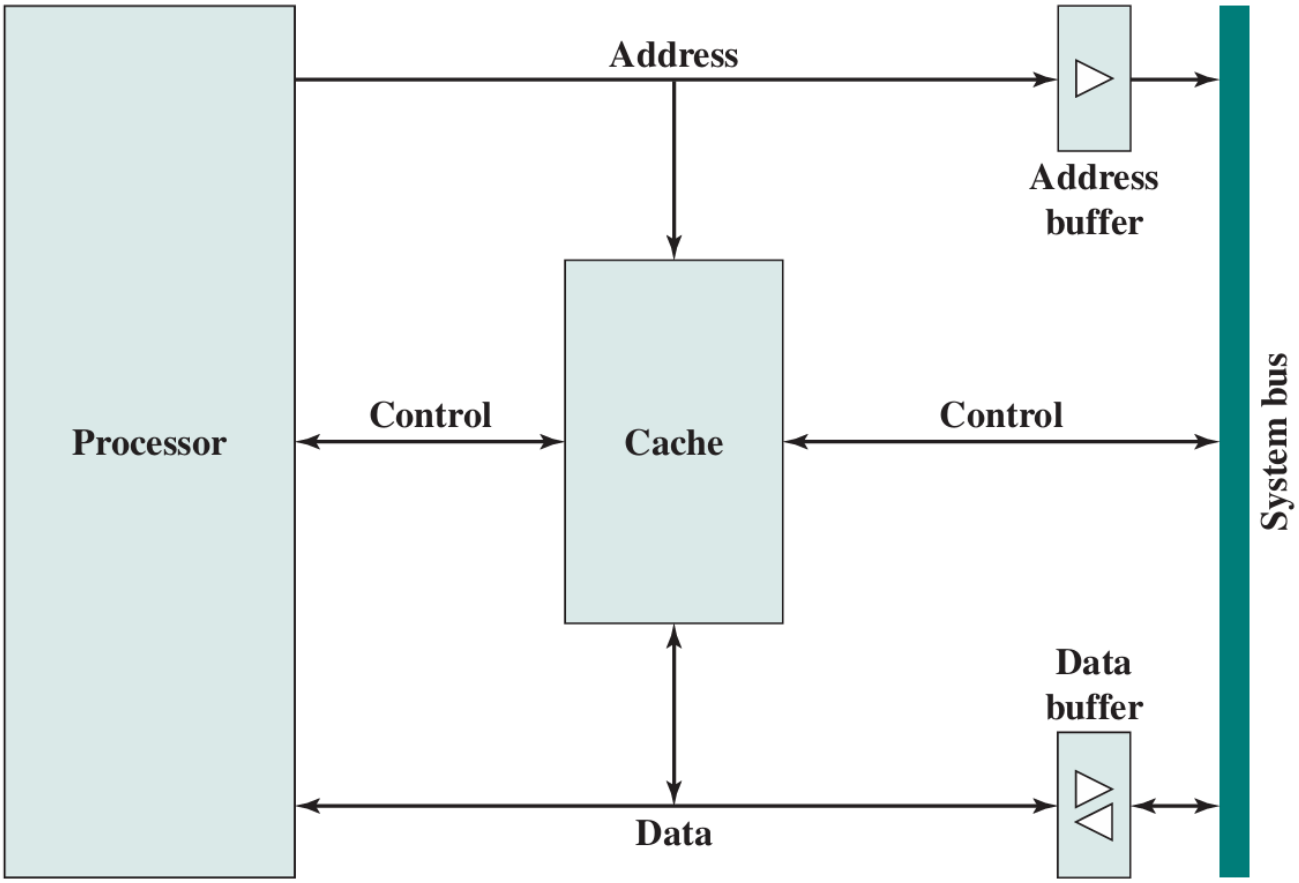


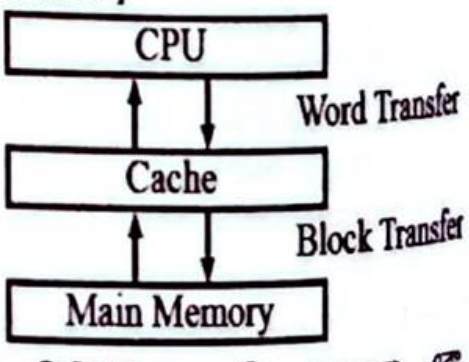

**Figure 4.6** Typical Cache Organization

**Extra info:**

## ক্যাশ মেমরি

প্রশ্ন ১ঃ ক্যাশ মেমরি কি? ক্যাশ মেমরির প্রকারভেদ লিখ? [অর্থ মন্ত্রনালয়-প্রোগ্রামার-১৩] [BPSC-ICT-ANE-20]

উত্তরঃ ক্যাশে মেমরি একটি বিশেষ অতি উচ্চ গতির মেমরি, যা আকারে ছোট তবে main memory (RAM) এর চেয়ে উচ্চগতি সম্পন্ন। CPU এর primary memory থেকে এটি আরও দ্রুত ডাটা অ্যাক্সেস করতে পারে। সুতরাং, এটি সিপিইউতে গতি বাড়ানো এবং



synchronize করতে এবং CPU এর কার্যকারিতা উন্নত করতে ব্যবহৃত হয়।

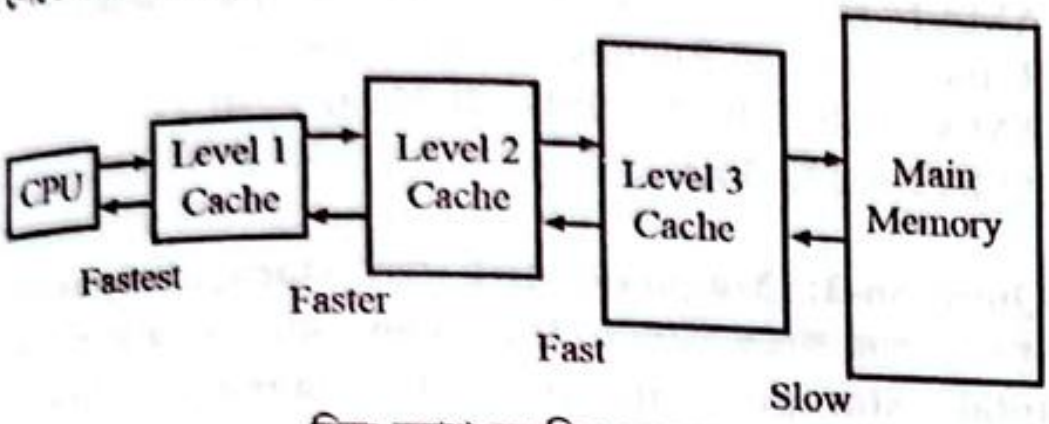
**চিত্র: ক্যাশ মেমরি**

ক্যাশ মেমরিকে তিনটি লেভেলে ভাগ করা যায়।

লেভেল- ১ ক্যাশ (প্রাইমারি ক্যাশ)

লেভেল-২ ক্যাশ (সেকেন্ডারি ক্যাশ)

লেভেল-৩ ক্যাশ (মেন মেমরি)




চিত্রঃ ক্যাশ মেমরির লেভেল

লেভেল- ১ ক্যাশ (প্রাইমারি ক্যাশ)ঃ লেভেল- ১ ক্যাশ হল প্রাথমিক ক্যাশ মেমরি। এটি 2KB থেকে 64KB সাইজের মধ্যে হয়ে থাকে এবং অন্যদের তুলনায় ক্যাশের আকার খুব ছোট যা কম্পিউটারের মাইক্রোপ্রসেসর উপর নির্ভর করে।

লেভেল-২ ক্যাশ (সেকেন্ডারি ক্যাশ)ঃ লেভেল-২ ক্যাশ হল সেকেন্ডারি ক্যাশ মেমরি। লেভেল-২ ক্যাশের আকার লেভেল- ১ ক্যাশ এর চেয়ে বেশি যা 256KB থেকে 512KB এর মধ্যে হয়ে থাকে যা কম্পিউটারের মাইক্রোপ্রসেসরে অবস্থিত। এটি হাই-স্পিড সিস্টেম বাসের সাথে মাইক্রোপ্রসেসরের আন্তঃসংযোগ করে থাকে।

লেভেল-৩ ক্যাশ (মেন মেমরি)ঃ লেভেল-৩ ক্যাশ আকারে বৃহত্তর তবে লেভেল- ১ ক্যাশ এবং লেভেল-২ ক্যাশ এর চেয়ে ধীরগতির হয়, এটির আকার 1MB থেকে 8MB এর মধ্যে হয়ে থাকে। একাধিক কোর প্রসেসরগুলিতে প্রতিটি কোরের জন্য পৃথক লেভেল- ১ ক্যাশ এবং লেভেল-২ ক্যাশ থাকতে পারে তবে সমস্ত কোর একটি সাধারণ লেভেল-৩ ক্যাশ থাকে। র‍্যামের চেয়ে লেভেল-৩ ক্যাশের স্পিড দ্বিগুণ হয়ে থাকে।

প্রশ্ন ৪ঃ Cache Memory এর সাথে main Memory এর পার্থক্য কী? [BPSC-ICT-ANE-20]

Cache Memory এবং Main Memory এর মধ্যে পার্থক্য নিম্নরূপঃ

| Cache Memory | Main Memory |
|---|---|
| ইহা দ্রুত গতি সম্পন্ন। | ইহা Cache Memory'র চেয়ে তুলনামূলক ধীরগতি সম্পন্ন। |
| ইহা আকারে ছোট। | ইহা আকারে বড়। |
| ইহা ব্যয়বহুল। | ইহা তুলনামূলক কম ব্যয়বহুল। |
| CPU তে কোন কিছু execution এর সময় যে data বারবার ব্যবহার করে ইহা সেটি ধারণ করে। | CPU তে যে program বা data execute হয় ইহা সেটি ধারণ করে। |
| ইহা সাধারণত Internal মেমরি। | ইহা Internal বা External উভয়ই হতে পারে। |
| এর সাইজকে তিনটি লেভেলে ভাগ করা যায়। যথাঃ<br>L1: 2KB-64KB<br>L2: 256KB-512KB<br>L3: 1MB-8MB | এর সাইজ সাধারণত 1GB, 2GB, 4GB, 8GB, 16GB হয়ে থাকে। |

***Cache Memory – necessary links***

https://www.geeksforgeeks.org/computer-organization-architecture/simultaneous-and-hierarchical-cache-accesses/

https://testbook.com/question-answer/the-access-time-of-cache-memory-is-100-ns-and-that--5fbf4e3bea3c7a9235587074

**AMAT for 3 Level memory hierarchy.**

**AMAT = H1*T1 + (1-H1)*H2*(T1 + T2) + (1-H1) *(1-H2)*(T1+T2+T3)**

**= T1 + (1-H1)*T2 + (1-H1)*(1-H2)*T3 :::::need to prove**

Example: https://gateoverflow.in/2308/gate-cse-1993-question-11

https://www.youtube.com/watch?v=MKvBu4eh4jg